

Documento de Proposta Técnica — TennisSync

Projeto Interdisciplinar — 1ª Entrega Equipe: TennisSync **Data:** 25/09/2026

1. Descrição Técnica do Problema e Delimitação do Escopo

1.1. Contextualização do Problema

No treinamento de tênis convencional, a correção postural e biomecânica dos golpes depende exclusivamente da observação contínua de um treinador humano. Em cenários onde o treinador não está presente ou quando se deseja analisar o desempenho com maior precisão, recorre-se à gravação de vídeos para análise manual pós-treino — um processo lento, subjetivo e que impossibilita ajustes instantâneos durante a prática.

Essa limitação cria uma lacuna significativa: o atleta executa repetidamente movimentos com vícios posturais sem receber correção em tempo real, o que pode consolidar padrões biomecânicos prejudiciais e aumentar o risco de lesões por esforço repetitivo.

1.2. Solução Proposta

O **TennisSync** é um sistema de análise biomecânica em tempo real para jogadores de tênis, desenvolvido em Python, que automatiza o processo de diagnóstico postural durante o treino. O sistema utiliza visão computacional e inteligência artificial para:

- **Capturar** a pose corporal do atleta via câmera, utilizando o MediaPipe Pose Landmarker para extrair 33 landmarks tridimensionais (x, y, z) do corpo;
- **Classificar** o tipo de golpe em execução (Forehand, Backhand ou Saque) através de um classificador temporal baseado em Transformer, que analisa uma janela deslizante de 60 frames;
- **Diagnosticar** erros posturais em tempo real por meio de um motor de regras geométricas que calcula os ângulos articulares utilizando trigonometria 3D (produto escalar de vetores) e os compara com limiares biomecânicos pré-definidos.

1.3. Delimitação do Escopo

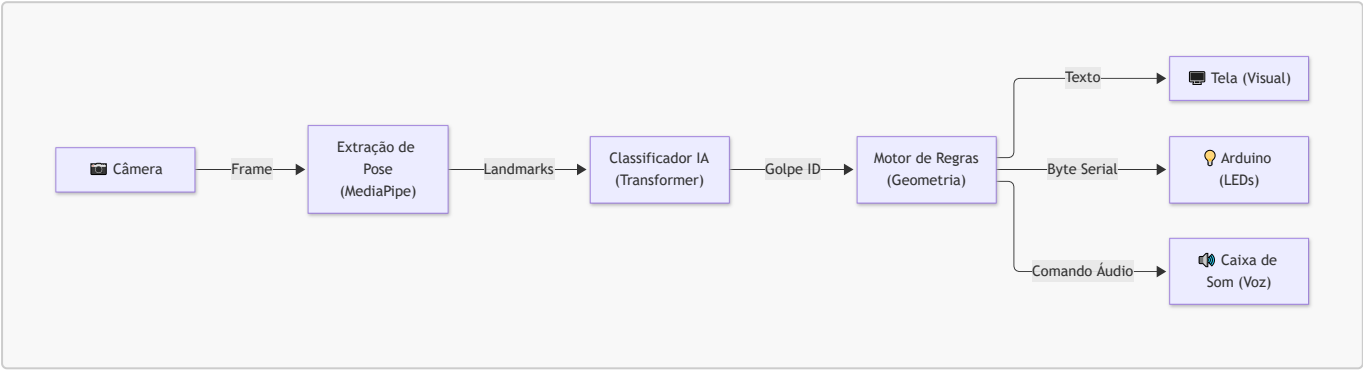
O escopo da solução delimita-se a:

- **Entrada:** Captura de vídeo em tempo real de um único jogador via câmera (OpenCV);
- **Processamento:** Extração de pose (MediaPipe), classificação temporal de golpes (Transformer) e diagnóstico geométrico (Motor de Regras);
- **Saída — Feedback Visual (Tela):** Texto renderizado sobre o vídeo via OpenCV indicando o golpe detectado e os erros identificados;
- **Saída — Feedback IoT (Painel Físico):** Painel de 12 LEDs integrado via Arduino, onde cada LED representa uma articulação corporal, acionado por comunicação Serial para indicar exatamente qual articulação falhou;
- **Saída — Feedback Auditivo (Voz):** Caixa de som acoplada ao circuito que verbaliza a instrução corretiva adequada.

Fora do escopo: Análise de múltiplos jogadores simultâneos, detecção de golpes além de Forehand/Backhand/Saque, armazenamento histórico de sessões e interface gráfica dedicada.

2. Diagrama de Blocos do Sistema

A arquitetura modular do TennisSync é dividida em três camadas de processamento isoladas. Esta separação garante que novas regras de correção postural possam ser adicionadas sem necessidade de retreinar o modelo de inteligência artificial.



Legenda dos Componentes

Componente	Tipo	Descrição Completa
Câmera (OpenCV)	Sensor	Captura de vídeo em tempo real a 30 fps.
MediaPipe Pose	Módulo Process.	Extrai 33 landmarks 3D (x,y,z) e centraliza no quadril.
TennisTransformer	Módulo IA	Buffer de 60 frames. Classifica golpe com confiança > 0.85.
Motor de Regras	Módulo Process.	Diagnóstico geométrico por trigonometria (produto escalar).
Tela (OpenCV)	Atuador	Feedback visual textual no vídeo (exibição por 2s).
Arduino (12 LEDs)	Atuador	Painel físico de feedback para cada articulação afetada.
Caixa de Som	Atuador	Instrução corretiva por voz focada na articulação defeituosa.

3. Definição do Fluxo de Dados

O ciclo de vida da informação segue o modelo **Entrada** → **Processamento** → **Saída** → **Registro**, operando de forma contínua e em tempo real:

3.1. Entrada

A câmera, acessada via `cv2.VideoCapture(1)`, fornece frames de vídeo contínuos no formato BGR. Cada frame é convertido para o espaço de cor RGB através do OpenCV e encapsulado em um objeto para inferência.

3.2. Processamento

1. **Extração de Pose:** O MediaPipe analisa o frame e extrai as coordenadas (x, y, z) de 33 landmarks corporais. Estas coordenadas são centralizadas no quadril para invariância de posição espacial.

- 2. **Classificação Temporal:** As coordenadas de cada frame são inseridas em um buffer deslizante de 60 frames. O tensor gerado é submetido ao modelo `TennisTransformerGPU`, que retorna o par `(class_id, confidence)`.
- 3. **Diagnóstico Geométrico:** Se a confiança for superior a 85% e o golpe for válido, o motor de regras calcula os ângulos articulares usando produto escalar de vetores e os compara com os limiares corretos.

3.3. Saída e Registro

O sistema emite feedback simultâneo por três canais: exibe texto na tela do vídeo, envia bytes (ex: `D`) via Serial ao Arduino para acender LEDs, e verbaliza instruções pela caixa de som. Após alertar, o sistema remove os últimos 20 frames do buffer (**cooldown**) para evitar falsos alertas em um mesmo movimento, e envia o byte `R` para o Arduino apagar os painéis.

4. Requisitos Funcionais e Não Funcionais

4.1. Requisitos Funcionais (RF)

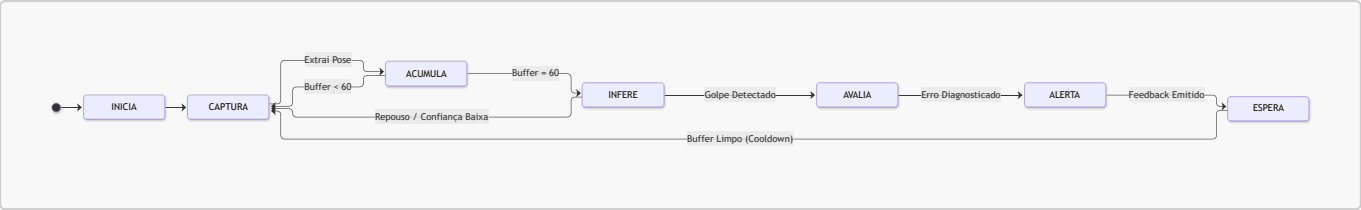
ID	Requisito	Descrição
RF01	Extração de Landmarks	Extrair coordenadas 3D (x, y, z) de 33 landmarks e centralizá-las no quadril.
RF02	Classificação Temporal	Classificar golpes e repouso utilizando buffer deslizante de 60 frames.
RF03	Cálculo de Ângulos	Calcular ângulos articulares usando produto escalar (trigonometria 3D).
RF04	Diagnóstico por Regras	Identificar condições de erro (ex: cotovelo flexionado, joelhos não dobrados).
RF05	Comunicação Serial	Enviar byte referencial ao Arduino para acionar LED específico.
RF06	Feedback na Tela	Renderizar diagnóstico e confiança no frame de vídeo por 2 segundos.
RF07	Feedback por Voz	Reproduzir instrução corretiva por áudio ao identificar um erro.
RF08	Reset IoT	Enviar comando <code>R</code> (Reset) ao Arduino para apagar os LEDs.

4.2. Requisitos Não Funcionais (RNF)

ID	Requisito	Descrição
RNF01	Protocolo de Comunicação	Operar em Serial a 9600 baud na porta COM4.
RNF02	Modularidade	Novas regras não podem exigir retreinamento da IA (Camadas isoladas).
RNF03	Desempenho Real-time	Processamento assíncrono para desacoplar CPU (OpenCV) e GPU (Transformer).
RNF04	Tolerância a Falhas	Continuar funcionando por software caso o Arduino seja desconectado.

5. Modelagem do Comportamento (Máquina de Estados)

O ciclo do sistema é modelado como uma máquina de estados finitos (7 estados):



6. Product Backlog Inicial (Framework Scrum)

Épico 1: Infraestrutura e Rastreamento Corporal

ID	User Story	Status
US01	Configurar o MediaPipe para extrair coordenadas e centralizar pelo quadril.	☒ Concluído
US02	Implementar filtro de suavização para reduzir <i>jittering</i> das coordenadas.	☐ Pendente

Épico 2: Inteligência de Contexto (Classificação Temporal)

ID	User Story	Status
US03	Construir modelo transformer mock para gerar previsões de teste.	☒ Concluído
US04	Integrar modelo real treinado para prever golpes baseados no dataset final.	☐ Pendente
US05	Desacoplar OpenCV da GPU via multi-threading para manter FPS estável.	☐ Pendente

Épico 3: Motor Biomecânico e Geometria

ID	User Story	Status
US06	Implementar cálculo de produto escalar de vetores 3D para ângulos.	☒ Concluído
US07	Adicionar configuração de lateralidade para espelhar alertas de canhotos.	☐ Pendente

Épico 4: Feedback Multissensorial de Hardware (IoT e Áudio)

ID	User Story	Status
US08	Codificar firmware em C++ no Arduino para ler porta Serial e mapear LEDs.	☒ Concluído
US09	Integrar pacote <code>PySerial</code> no Python para envio do gatilho e reset.	☒ Concluído
US10	Adicionar caixa de som conectada ao circuito para verbalizar correção (Voz).	☐ Pendente

Documento de Proposta Técnica gerado em 25/09/2026 como parte da 1ª Entrega.